genfleet

GUC German University in Cairo الجامعة الألمانية بالقاهرة

Kestrel Telemetry Pack v1: Dataset Description

Companion to the **Kestrel** project description. This document is the field-level contract for `kestrel-telemetry-pack-v1.zip`. Writing the machine-readable contracts (Pydantic, Pandera) from it is part of Milestone 2.

All data is synthetic. Tenant, agent, person, and host names are fictional. No production logs, no credentials, no personal data, and no working exploit payloads appear anywhere in the pack.

0. Conventions

RULE	VALUE
Canonical timestamps	UTC, ISO-8601 with explicit offset (`2026-03-14T09:22:01.412Z`). Not every source follows this: find out which do not
Identifier style	`tn_*` tenant, `ag_*` agent, `tl_*` tool, `sp_*` sandbox profile, `ss_*` session, `ev_*` event, `nf_*` network flow, `al_*` alert, `tk_*` task
Encoding	UTF-8, `\n` line endings, RFC 4180 CSV with a header row; Parquet with zstd compression
Nulls	empty field in CSV, `null` in JSON and Parquet, `-` in the syslog file
Booleans	`true` / `false` lowercase
Money	USD, 6 decimal places
Ordering	files are **not** sorted by time and must not be assumed to be

Two timestamps exist on every activity record and mean different things:

- `event_time` is when the thing happened, on the emitting host.
- `ingest_time` is when the Kestrel platform received it. **Lateness = `ingest_time - event_time`.**

Watermarks are defined on `event_time`. Never on `ingest_time`.

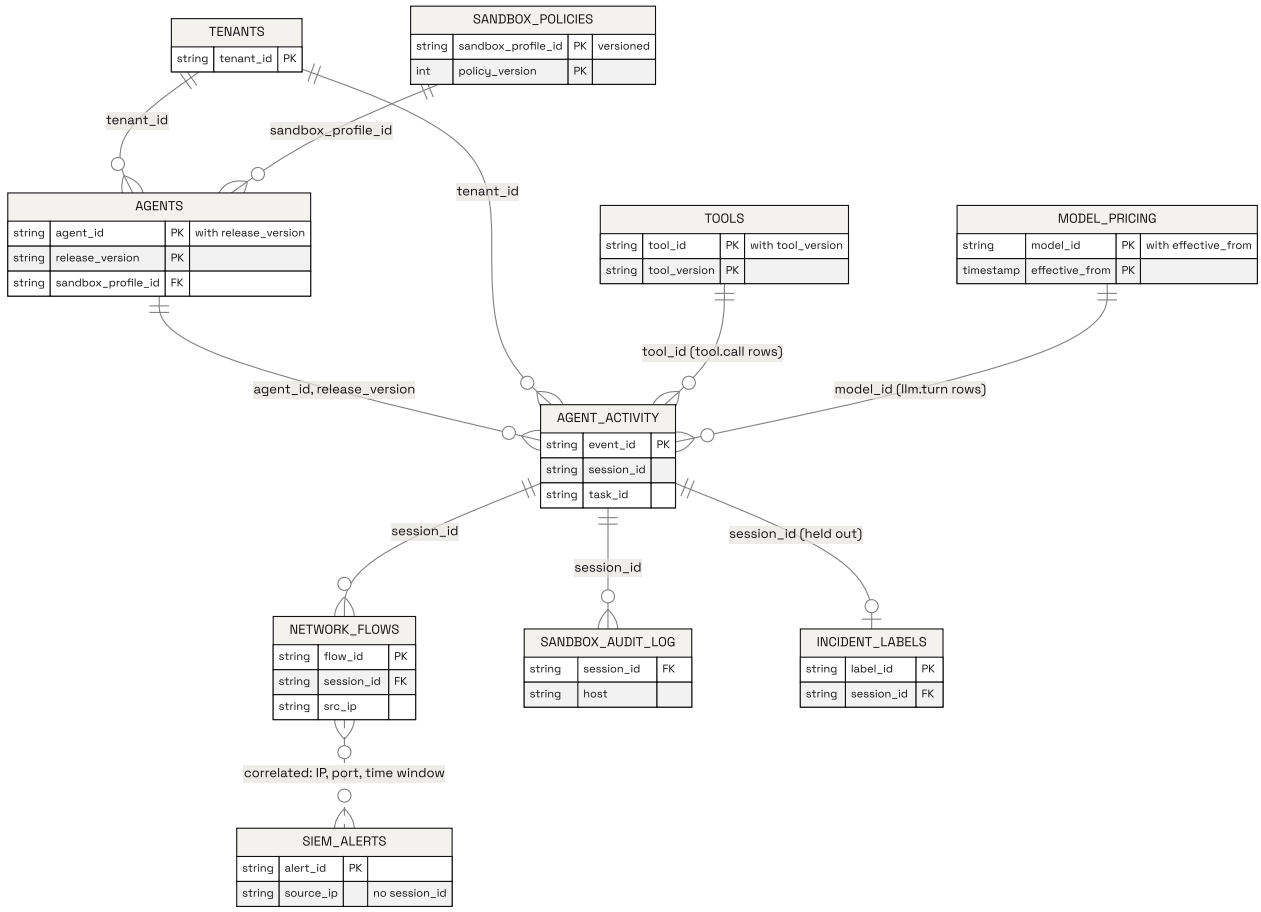
1. File inventory

#	FILE	FORMAT	GRAIN	APPROX. ROWS (DEFAULT SCALE)
1	`tenants.csv`	CSV	one tenant	6
2	`agents.csv`	CSV	one agent deployment version	22
3	`tools.csv`	CSV	one tool version	18
4	`sandbox_policies.json`	nested JSON	one policy version	10

#	FILE	FORMAT	GRAIN	APPROX. ROWS (DEFAULT SCALE)
5	`agent_activity.parquet`	Parquet	one activity event: lifecycle, LLM turn, or tool call	~940,000
6	`network_flows.csv`	CSV	one egress attempt	~42,000
7	`sandbox_audit.log`	syslog text	one runtime decision	~184,000
8	`siem_alerts.json`	nested JSON	one alert from a targeted system	~3,800
9	`model_pricing.csv`	CSV	one model price point	5
10	`incident_labels.csv`	CSV	one labelled session	~6,000
11	`stream_events.json`	JSON array	one live activity event (hold-out)	~9,600

The pack covers 60 days (2026-01-31 to 2026-03-31) of batch data; the stream replays the first half hour of 2026-04-01. Build dates and hour bands (`dim_date` , `dim_time`) yourself from the timestamps: no calendar file is supplied.

2. Entity relationship



`session_id` is the join key across files 5 to 7. It is **absent from `siem_alerts.json`**, because that feed comes from the *targeted* systems, which have never heard of Kestrel sessions. Correlation is part of the work.

3. Field dictionary

3.1 `tenants.csv`: the companies using the platform

COLUMN	TYPE	NOTES
`tenant_id`	string PK	`tn_*`
`tenant_name`	string	fictional company name
`industry`	enum	`finance`, `healthcare`, `logistics`, `retail`, `public_sector`
`plan_tier`	enum	`starter`, `growth`, `enterprise`; governs budgets
`region`	enum	`eu-west-1`, `us-east-1`, `me-south-1`
`onboarded_at`	timestamp	
`is_active`	boolean	some inactive tenants still have historical traffic
`security_contact`	string	fictional role mailbox

3.2 `agents.csv`: agent metadata

COLUMN	TYPE	NOTES
`agent_id`	string	`ag_*`; natural key is `(agent_id, release_version)`
`release_version`	string	semver
`agent_name`	string	
`agent_type`	enum	`research`, `support`, `ops`, `analytics`, `coding`
`model_id`	string	FK → `model_pricing.model_id`
`tenant_id`	string	FK → `tenants`
`sandbox_profile_id`	string	FK → `sandbox_policies`
`owner_team`	string	
`autonomy_level`	enum	`supervised`, `semi_autonomous`, `autonomous`
`max_tool_calls_per_session`	int	declared budget
`launch_date`	date	
`is_deprecated`	boolean	

3.3 `tools.csv` : tool catalogue

COLUMN	TYPE	NOTES
`tool_id` , `tool_version`	string	composite natural key
`tool_name`	string	the name agents and logs use for the tool
`tool_category`	enum	`retrieval` , `database` , `shell` , `network` , `filesystem` , `messaging` , `code_exec`
`risk_tier`	enum	`low` , `medium` , `high` , `critical`
`requires_approval`	boolean	
`owner_team`	string	

3.4 `sandbox_policies.json` : the rules an agent is supposed to obey

Nested JSON, an array of policy-version objects. Requires flattening.

```
{
  "sandbox_profile_id": "sp_research_std",
  "policy_version": 3,
  "valid_from": "2026-02-01T00:00:00Z",
  "valid_to": null,
  "allowed_tools": ["tl_web_fetch", "tl_vector_search", "tl_summarize"],
  "network": {
    "egress_allowlist": ["api.kestrel.ai", "*.public-data.example"],
    "denied_ports": [22, 23, 445, 3389],
    "max_egress_mb": 50
  },
  "filesystem": { "read_scope": ["/workspace"], "write_scope": ["/workspace/out"] },
  "resources": { "max_runtime_s": 900, "max_memory_mb": 2048 },
  "escalation_allowed": false
}
```

Allowlist entries can be wildcards (`*.public-data.example`) and must be matched, not string-compared. A policy version with no `network` block allows no egress at all.

This file is the spine of the project. "Policy violation" is only computable by joining an activity record to the policy version whose validity window contains the record's `event_time` .

3.5 `agent_activity.parquet` : what every agent did

One row per activity event. Counts are typed `int64` ; the three timestamp columns are stored as raw strings, exactly as the platform exported them, so you must parse them yourself. The row's `event_type` decides which columns are filled; the rest are empty. A session's rows run `session.start` (which carries the assigned task), then alternating `llm.turn` and `tool.call` rows, then one terminal row.

COLUMN	TYPE	FILLED ON	NOTES
`event_id`	string PK	all	`ev_*`

COLUMN	TYPE	FILLED ON	NOTES
`event_time`, `ingest_time`	timestamp	all	stored as raw strings
`sequence_number`	int	all	per session, from 1
`event_type`	enum	all	`session.start`, `llm.turn`, `tool.call`, `session.end`, `session.error`
`session_id`, `tenant_id`, `agent_id`, `release_version`	string	all	FK
`sandbox_instance_id`	string	all	the running container
`status`	enum	all	`ok` / `error` on lifecycle and turn rows; `success`, `failure`, `denied`, `timeout` on tool calls
`latency_ms`	int	turn, tool, terminal	**per-event only**
`turn_index`	int	`llm.turn`	0-based within the session
`model_id`	string	`llm.turn`	FK → `model_pricing`
`content_source`	enum	`llm.turn`	`user`, `system`, `retrieved_web`, `tool_output`: where the newest content in the context came from; indirect injection arrives as `retrieved_web` or `tool_output`
`stated_goal`	string	`llm.turn`	the goal the agent articulated this turn; the drift signal
`injection_indicators`	string	`llm.turn`	pipe-separated enum, empty when none: `instruction_override`, `role_confusion`, `exfil_request`, `encoded_payload`, `tool_coercion`, `policy_probe`
`input_tokens`, `output_tokens`	int	`llm.turn`	
`finish_reason`	enum	`llm.turn`	`stop`, `length`, `tool_call`, `content_filter`
`tool_id`, `tool_version`	string	`tool.call`	FK → `tools`
`finished_at`	timestamp	`tool.call`	the call's start is `event_time`
`retry_number`	int	`tool.call`	**0 on the first attempt**
`error_class`	string	`tool.call`	`TIMEOUT`, `AUTH_DENIED`, `RATE_LIMIT`, `POLICY_DENIED`, `TARGET_UNREACHABLE`, `INVALID_ARGS`

COLUMN	TYPE	FILLED ON	NOTES
`target_resource`	string	`tool.call`	host, table, path, or command touched; arguments themselves are never stored
`bytes_out`	int	`tool.call`	
`task_id`	string	`session.start`	`tk_*`; the task this session was started for
`task_category`	enum	`session.start`	`market_research`, `ticket_triage`, `report_generation`, `data_query`, `code_review`, `incident_response`
`expected_tool_categories`	string	`session.start`	pipe-separated, e.g. `retrieval|messaging`; what the task *should* need, and the baseline for **goal drift**

> End-to-end session duration is **derived**: terminal `event_time` − `session.start` `event_time`. `latency_ms` is a cross-check only; report how often the two disagree.
>
> `injection_indicators` is a **detector output, not ground truth**: it has false positives and misses. `incident_labels.csv` is the ground truth, and is held out.

3.6 `network_flows.csv`: egress attempts

COLUMN	TYPE	NOTES
`flow_id`	string PK	`nf_*`
`session_id`, `sandbox_instance_id`	string	FK
`flow_time`	timestamp	
`src_ip`	string	the sandbox's address; the correlation key to the SIEM alerts (section 3.8)
`dst_host`, `dst_ip`, `dst_port`	string / int	
`protocol`	enum	`https`, `http`, `ssh`, `smb`, `dns`, `rdp`
`bytes_sent`, `bytes_received`	int	
`verdict`	enum	`allowed`, `blocked`, `logged_only`
`geo_country`	string, nullable	

Flows are only written for tool-initiated egress; LLM calls go through the platform gateway and do not appear.

3.7 `sandbox_audit.log`: runtime decisions, syslog text

One line per decision, **host-local time with no offset**. Must be regex-parsed and normalised.

```
Mar 14 11:22:04 sbx-eu-113 kestrel-sandbox[8841]: session=ss_00042 instance=sbx_9f31 \
  decision=DENY action=socket_connect target=10.20.4.19:445 reason=EGRESS_NOT_ALLOWED
policy_version=3
```

Parsed fields: `log_time` (naive), `host`, `process`, `pid`, `session_id`, `sandbox_instance_id`, `decision` (`ALLOW` | `DENY`), `action` (`socket_connect`, `file_read`, `file_write`, `process_spawn`, `setuid`, `mount`, `ptrace`), `target`, `reason`, `policy_version`.

The hosts' offset from UTC is not written anywhere. Work it out from records you can line up with other sources, and state how you did it.

3.8 `siem_alerts.json`: what the targeted systems saw

A SIEM (*security information and event management* system) collects logs from a company's machines and raises alerts on suspicious activity. These alerts come from the SIEMs of the systems the agents reached, not from Kestrel.

A nested JSON export: an envelope with an `alerts` array. Requires flattening.

```
{"generated_at": "2026-04-01T02:00:00+02:00", "source": "corp-siem-export-v2", "alerts": [
  {"alert_id": "al_00007", "source_system": "corp-siem", "severity": "high",
   "detected_at": "2026-03-14T13:22:41+02:00", "source_ip": "10.20.9.113",
   "destination": {"host": "10.20.4.19", "port": 445},
   "technique": {"id": "T1021.002", "name": "SMB/Windows Admin Shares"},
   "rule": "lateral_movement_smb_burst", "confidence": 0.82}]}
```

`severity` is `low` … `critical`. `destination.host` is an IP address.

No `session_id`. Correlate to `network_flows` on `(src_ip = source_ip, dst_ip/port, |Δtime| ≤ N)`. Declare your `N`, and report matched, ambiguous (multi-match), and unmatched counts. Some alerts are genuinely unmatchable, because they come from activity outside the platform. Do not force them.

3.9 `model_pricing.csv`

`model_id`, `model_family`, `input_price_per_1k`, `output_price_per_1k`, `currency`, `effective_from`, `effective_to`. Cost uses the price effective **at turn time**.

3.10 `incident_labels.csv`: held-out ground truth

COLUMN	TYPE	NOTES
`label_id`	string PK	
`session_id`	string	FK, unique
`label`	enum	`benign`, `suspicious`, `confirmed_rogue`
`attack_stage`	enum, nullable	`injection`, `recon`, `privilege_escalation`, `lateral_movement`, `exfiltration`
`technique_ids`	string	pipe-separated MITRE ATT&CK IDs
`labelled_by`	enum	`analyst`, `automated`, `red_team`
`labelled_at`	timestamp	

COLUMN	TYPE	NOTES
`notes`	string	short synthetic analyst note

Sparse by design: about 6% of sessions are labelled, of which a minority are rogue. **Never join this into a feature.** Use it once, at the end, to report precision and recall.

3.11 `stream_events.json` : Kafka hold-out feed

A JSON array of records with the same fields as `agent_activity.parquet` , `null` for empty fields. Your producer publishes one element per Kafka message. It is ordered by `ingest_time` and ends with a control record:

```
{"event_type": "EOS", "event_time": "2026-04-01T00:31:12.000Z", "producer_id": "replay-1"}
```

Expect the feed to be as untidy as the batch files. Every record your consumer refuses carries exactly one reason from the closed enum in section 4.

4. Rejection reason enum (closed)

`DUPLICATE_EVENT_ID` , `UNKNOWN_AGENT_ID` , `UNKNOWN_TENANT_ID` , `MALFORMED_TIMESTAMP` , `ORPHAN_EVENT` , `INVALID_EVENT_TYPE` , `MISSING_REQUIRED_FIELD` , `INVALID_NUMERIC_RANGE` , `LATE_BEYOND_WATERMARK` , `UNPARSEABLE_RECORD` , `SCHEMA_VIOLATION` .

Exactly one per rejected record. If two apply, use the first in this order and say so in your data dictionary. `ORPHAN_EVENT` is any non-start event whose session has no accepted `session.start` .

5. Canonical activity model (the required output shape)

Every source, whatever its original format, normalises into one row shape before modelling:

FIELD	SOURCE OF TRUTH
`activity_id`	the source record's PK, prefixed by source
`source_file` , `source_format`	provenance, must survive to the warehouse
`event_time_utc` , `ingest_time_utc` , `lateness_s`	normalised
`tenant_id` , `agent_sk` , `session_id` , `task_id`	resolved FKs; `agent_sk` is the Type-2 surrogate key current at `event_time_utc`
`activity_type`	`lifecycle` | `prompt_turn` | `tool_call` | `network_flow` | `sandbox_decision` | `siem_alert`
`activity_subtype`	the source's own event/action/decision name
`status` , `verdict`	normalised outcome
`target_resource`	host, table, path, or tool
`policy_version_applied` , `is_policy_violation`	derived by validity-window join

FIELD	SOURCE OF TRUTH
`risk_indicators`	pipe-separated, union of injection indicators, denials, and violations
`tokens_in`, `tokens_out`, `cost_usd`	prompt turns only
`quality_flags`	pipe-separated, non-fatal issues retained rather than silently fixed

Records that cannot be normalised go to `fact_rejected_record` with their raw payload, source file, line number, and exactly one enum reason. **Nothing is ever silently dropped.**

6. How the pack is made

The pack is generated by a simulator with a fixed random seed, so every team receives identical files. About 1.5% of sessions are rogue. Traffic is not evenly spread across tenants and agents: finding out where it concentrates is part of the Spark work.

Appendix A: Required metric definitions

The Project Description lists the required metrics by name. Their definitions follow. For each, your data dictionary must still state the denominator, population filter, late-event policy, and null policy.

METRIC	DEFINITION
Session volume · success · error rate	distinct `session_id` with a terminal event; successful, resp. error, terminal sessions / all terminal
p50 / p95 session duration	percentile of derived end-to-end duration for completed sessions
Policy- · egress-violation rate	tool calls, resp. flows, outside the **effective** allowlist / all
Sandbox denial rate · escalation attempts	`DENY` / all decisions; sessions with ≥1 denied escalation-class syscall
Goal-drift · injection-indicator rate	sessions whose executed tool categories diverge from the task's expected set; turns with ≥1 indicator / all turns
Tool failure · retry rate	failed / completed attempts; attempts with `retry_number > 0` / all
Blast radius · time-to-first-violation	distinct destination hosts per rogue session; first violation − first indicator turn
Trace completeness · correlation coverage	terminal sessions with the required lifecycle events; SIEM alerts matched to a session
Rejected · late-event rate	rejected / received; late-accepted / accepted
Detection precision / recall	your rule output scored against `incident_labels.csv`
Estimated LLM cost	`(in_tokens/1000 × in_price) + (out_tokens/1000 × out_price)` at the price effective at turn time

Conventions, so that every team computes the same quantities:

- **Deduplication:** on `event_id`, keeping the copy with the earliest `ingest_time`.

- **Late event:** `ingest_time − event_time` greater than 10 seconds.
- **Terminal session:** one with a `session.end` or `session.error` row; duration percentiles use terminal sessions only.
- **Trace complete:** a terminal session with at least one `llm.turn` and no gap in its accepted `sequence_number`s.
- **Goal drift denominator:** sessions with `expected_tool_categories` and at least one accepted tool call.
- **Time to first violation:** from the session's first turn carrying an injection indicator to the first tool-policy or egress violation at or after it.
- **Percentiles:** linear interpolation between closest ranks; in Spark, name the function and its relative error.
- **Missing token counts:** your choice, declared, with its effect shown on one metric.

Appendix B: Kafka topology

The required topic layout for Milestone 2.

SETTING	VALUE
Topics	`kestrel.agent.events` (raw) · `...curated` · `...rejected`
Partitions / key	3 on raw; keyed by `session_id` so one session keeps relative order
Consumer group	one per team; offsets committed **after** the write succeeds
Delivery / watermark	at-least-once, hence mandatory `event_id` dedup; 60 s lateness on `event_time`

Appendix C: Spark analysis matrix

The ten required Milestone 2 analyses: five in the DataFrame API, five in Spark SQL.

DATAFRAME API	SPARK SQL
Daily session volume and policy-violation rate by agent/version	Top five agents by p95 duration, with a minimum session threshold
Approximate p50/p95 duration by agent type and hour band	Tool categories with the highest failure and denial rates
Token, cost, and turn distribution by agent/version and tenant	Agent versions with the largest week-over-week violation-rate increase
Per-session escalation chain: ordered violations with time deltas (window)	Sessions with the most tool attempts before a terminal event
Trace-completeness, late-event, rejected-record rate by event type	Detection precision/recall by tenant and agent type